Calculating complex roots of a polynomial with real coefficients using the Bairstow's method.

1. Application

Function *Bar* calculate all complex and real roots of given polynomial. User is provided a choose between 3 different representations: floating point, floating point converted to intervals, intervals.

2. Description of the method

Having a polynomial $P(x) = \sum_{i=0}^{n} a_i x^i$ we can divide it by quadratic equation $D(x) = x^2 + ux + v$. That way:

$$P(x) = \left(D(x) * \sum_{j=0}^{n-2} b_j x^j \right) + (nx + m)$$

If $nx + m = 0$, roots of quadratic equation are roots of polynomial.

Finding right quadratic equation is iterative way. It starts by performing first synthetic division of $P(x)$:

$$P(x) = (x^2 + ux + v)\left(\sum_{i=0}^{n-2} b_i x^i \right) + (nx + m)$$

$$B(x) = \left(\sum_{i=0}^{n-2} b_i x^i \right)$$

and again, performing a synthetic division on $B(x)$ dividing by $D(x)$:

$$B(x) = (x^2 + ux + v)\left(\sum_{i=0}^{n-4} c_i x^i \right) + (gx + h)$$

$$C(x) = \left(\sum_{i=0}^{n-4} c_i x^i \right)$$

And

$$b_i = \begin{cases} 0 & i = n, n-1 \\ a_{i+2} - ub_{i+1} - vb_{i+2} & i = n-2, \dots, 0 \end{cases}$$

what also means that:

$$n = a_1 - ub_o - vb_1$$
$$m = a_0 - vb_o.$$

Similar for second division:

$$c_i = \begin{cases} 0 & i = n, n-1 \\ b_{i+2} - uc_{i+1} - vc_{i+2} & i = n-2, \dots, 0 \end{cases}$$

$$g = b_1 - uc_0 - v_{c1}$$
$$h = b_0 - vc_0.$$

The quadratic evenly divides the polynomial when:

$$c = 0 \; and \; d = 0$$

while c and d can be written as a function of u and v:

$$c(u, v) = 0 \quad and \quad d(u, v) = 0$$

what allows using iterating Newton–Raphson method what can be briefly written as:

$$\Delta u = \frac{b_n c_{n-3} - b_{n-1}\, c_{n-2}}{c_{n-2}^2 - c_{n-1} c_{n-3}}$$

and

$$\Delta v = \frac{b_{n-1} c_{n-1} - b_n\, c_{n-2}}{c_{n-2}^2 - c_{n-1} c_{n-3}}$$

That way we find quadratic equation that solves the polynomial, then solve the quadratic equation to find roots r_1, r_2:

$$r = \frac{u \pm \sqrt{u^2 + 4v}}{2}$$

with possible complex results.

That kind of iterations are performing as long as degree is bigger or equal 3.

This method was invented and firstly used by Leonard Bairstow in *Applied Aerodynamics* where he needs to find a root of polynomial that is 8th degree.

certain errors of operation which affect the solution to a large extent. The final result obtained in the present example is

$$\lambda^8 + 20{\cdot}4\lambda^7 + 151{\cdot}3\lambda^6 + 490\lambda^5 + 687\lambda^4 + 719\lambda^3 + 150\lambda^2 + 109\lambda + 6{\cdot}87 = 0 \quad . \quad (82)$$

This equation has two real roots only, which can be extracted if desired by Horner's process. A general method for all roots has been given by Graeffe, and as this does not appear in the English text-books an account of its application to (82), is given as an appendix to this chapter. By use of the method it was found that equation (82) has the factors

$$(\lambda^2+11{\cdot}25\lambda+35{\cdot}1)(\lambda^2-0{\cdot}006\lambda+0{\cdot}171)(\lambda+7{\cdot}79)(\lambda+0{\cdot}067)(\lambda^2+1{\cdot}33\lambda+2{\cdot}19)=0 \quad . \quad (83)$$

3. Function call

Bar (choice, precision, max_iter, polynomial, real_roots, complex_roots)

4. Inputs

choice – mode of number calculation; 'f' for floating; 's' for interval from single number; 'i' for interval calculation from interval input

precision – precision of comparison, -1 for maximum possible precision

max_iter - maximum integration that will be performed, -1 for no limit

polynomial - polynomial coefficients starting from a_n to a_0, for 'i' mode 2 floating-point number, for 's' and 'f' just floating-point number.

5. Outputs

real_roots – array of real roots of given polynomial; 'I' and 's' it is interval; 'f' floating-point number

complex_roots – array of complex roots of given polynomial; 'I' and 's' it is two intervals; 'f' floating-point numbers

7. Data Type

Integer*: max_iter*, *status*

vector: polynomial, real_roots, complex_roots

character*: choice*

long doule*: precision*

8. Custom data type

Vector polynomial- dynamic array of coefficients of a polynomial

Vector real_roots- dynamic array of real roots

Vector complex_roots – dynamic array of dynamic array containing real and imaginary part of complex number

9. Function code

All of the codes are publicly available here: https://github.com/TMPkl/Bairstow-s

10. Example

a) Polynomial:

$$[0{,}9; 1]x^3 + [-2{,}1; -2]x^2 + [-5; -4{,}9]x + [6{,}6]$$

Input:

-1 <- set the precision

-1 <- set the maximum iteration

0.9 1.0

-2.1 -2

-5 -4.9

6 6

Output:

[3.052133758492346e+00, 4.148089245674311e+00] szerokość: 1.096e+00

[-1.935646440475890e+00, -8.396909532939260e-01] szerokość: 1.096e+00

[9.000000000000000e-01, 1.000000000000000e+00] szerokość: 1.000e-01

b) Polynimial:

$$x^6 + 10x^5 + 34x^4 + 38x^3 - 73x^2 - 160x + 150$$

Input:

1e-15 <- precision

-1 <- maximum iteration

1

10

34

38

-73

-160

150

Output:

1.000000001654560e+00

-3.000000000000000e+00

9.999999983454404e-01

-5.000000000000000e+00

[-2.000000000000000e+00 + 2.449489742783178e+00i]

[-2.000000000000000e+00 + -2.449489742783178e+00i]

c) Polynomial:

$$x^3 + 5x^2 - 10x^1 + 100$$

Input:

1e-50

3

1

5

-10
100

Output:

(none output because iteration limit exceed)